

Rawabet: An AI-Powered Matching Platform Connecting the Lebanese Diaspora with Local Businesses

Gaelle Medawar · August 2026

1 Project Title & Abstract

Rawabet is a full-stack web platform that connects members of the Lebanese diaspora who wish to invest with local Lebanese businesses seeking funding, using a Tinder-style discovery interface. The core problem addressed is discovery: diaspora investors have capital and goodwill but no structured way to find businesses that match their niche, geographic, and budget preferences, while local businesses have no channel to reach diaspora capital.

Rawabet's methodology combines a conventional MERN-style web stack (MongoDB, Express, React, Node.js) with a large language model (Google Gemini) used not as a trained classifier but as a zero-shot compatibility scorer: investor and business profiles are sent in a single batched, schema-constrained prompt per deck load, and the model returns a 0–100 match score with a natural-language justification for each candidate. A deterministic rule-based fallback preserves availability when the API is rate-limited.

Across four independent automated test suites (over 50 assertions), the matching, real-time chat, notification, and directory-browsing features all function correctly, and qualitative samples show the AI's explanations are grounded in the specific mismatched or matched attributes of each pair. The project demonstrates that prompted, schema-constrained LLM calls are a viable, low-cost substitute for a trained recommendation model in a small-scale, two-sided marketplace.

2 Introduction & Problem Statement

Since 2019, Lebanon has experienced a severe and prolonged economic crisis, and the Lebanese diaspora — estimated at several times the size of the resident population — is frequently cited as

one of the country's most under-utilized economic resources. Diaspora members routinely express interest in supporting Lebanese entrepreneurship, whether out of sentiment, family ties, or genuine investment opportunity, but lack a structured way to discover businesses that match what they actually care about: a specific industry, a specific region of Lebanon, and an amount they are comfortable committing. Symmetrically, small Lebanese businesses seeking growth capital have no dedicated channel to reach this pool of willing diaspora capital; they are limited to informal networks, family connections, or generic crowdfunding platforms with no Lebanon-specific context.

Rawabet (“bonds” or “ties,” in Arabic) was built to address this discovery gap directly. Rather than a generic funding marketplace, the project's core hypothesis is that framing the problem as a two-sided matching problem — closer to a dating app than a listings site — produces a more engaging and more targeted discovery experience for both sides, and that an AI layer can meaningfully improve the quality of that matching beyond simple keyword or category filters.

The project's goals were therefore to:

- 1. Build a working two-sided platform** where investors and businesses each create a structured profile.
- 2. Use AI to rank and explain compatibility** between the two sides rather than relying on manual browsing alone.
- 3. Gate contact information behind mutual consent** (a “match”), mirroring the trust model of dating apps.
- 4. Support real-time communication** once a match forms.
- 5. Keep the system operable at zero marginal cost**, a constraint that directly shaped the AI provider, hosting, and storage choices described below.

3 Methodology

Rawabet's architecture separates cleanly into a Node.js/Express API, a MongoDB Atlas (free-tier) database accessed via Mongoose, and a React (Vite) single-page frontend, with Socket.io layered on the same HTTP server for real-time features. Six Mongoose collections model the domain: User (auth and role), InvestorProfile, BusinessProfile, Swipe (each like/pass decision), Match (created only once both sides have liked each other), and Message (chat history per match).

The AI matching component is the methodological core of the project. Rather than training or fine-tuning a model — infeasible at this scope, and without a labeled dataset of “good” investor–business pairs to train against — Rawabet uses a general-purpose foundation model (Google

Gemini, gemini-3.6-flash) as a zero-shot compatibility scorer via in-context prompting. This is best understood as a two-stage retrieve-then-rank pipeline, a common pattern in recommendation systems, but with a rule-based retriever and an LLM-based ranker in place of a learned embedding model at either stage:

- 1. Retrieval.** When an investor opens their discovery deck, the backend queries MongoDB for a bounded candidate pool (up to 15 businesses), biased toward the investor's stated niches and regions, falling back to unfiltered businesses if the biased pool is too small.
- 2. Ranking.** That pool, with the investor's full profile, is sent to Gemini in a single batched call. The prompt instructs the model to score every business 0–100 and justify each score, with the response constrained to a JSON Schema so the output is guaranteed machine-parseable — no regex extraction or retry-on-malformed-JSON logic is needed.
- 3. Fallback.** If the API key is missing, the call errors, or the free-tier rate limit is hit, a deterministic rule-based scorer (niche, region, and budget overlap) produces a score instead, so the deck never breaks.

Batching all candidates into one call, rather than one call per investor–business pair, was a deliberate cost and latency optimization: a deck load costs one API call regardless of pool size, at an estimated \$0.01 in equivalent paid pricing per load (free under Gemini's current free tier).

Beyond the AI-driven deck, a second discovery path — a filterable “Browse” directory (by niche, region, and amount range) — was implemented as a non-AI complement, reusing the same swipe endpoints so a user can express interest from either path. This was a deliberate response to a limitation of the retrieval stage: a business outside an investor's stated niche/region bias might still be worth discovering, and Browse guarantees full visibility regardless of the AI ranking.

4 Implementation Details & Results

The backend exposes a REST API for authentication (JWT, bcrypt-hashed passwords), profile management, the swipe/match lifecycle, and match history, plus a Socket.io layer for two real-time features: per-match chat rooms (joined only after the server verifies the connecting user is an actual participant in that match) and per-user personal rooms, joined automatically on connection, used to push a cross-page toast notification whenever a new message arrives on any of a user's matches, not just the one currently open. The frontend is a React SPA (Vite, React Router, Axios, socket.io-client) with role-aware navigation, a drag-to-swipe card interface built on native pointer events, and a live-updating chat view.

Because this is a systems and product-engineering project rather than a benchmark-driven ML project, results are reported as functional verification and qualitative demonstration rather than as an accuracy metric against a labeled test set — no such dataset exists for this problem. Four independent automated smoke-test suites were written against an isolated in-memory MongoDB instance and, for the real-time features, real socket connections against the running server.

4	50+	~\$0.01	11
TEST SUITES, ALL PASSING	AUTOMATED ASSERTIONS	PER BATCHED DECK SCORE	SEEDED DEMO PROFILES

The four suites covered: the core auth → onboarding → swipe → match flow (registration, role-based access control, AI-ranked deck retrieval with fallback scoring, mutual-swipe matching, contact reveal); real-time chat (bidirectional delivery, room-based authorization rejecting non-participants, REST/socket history consistency); cross-page notifications (correct recipient targeting, no self-notification); and the Browse directory’s filter and exclusion logic (niche, region, and amount-range filters; excluding already-matched profiles; a business never seeing its own profile in peer browsing).

Qualitative sample. For an investor profile focused on Food & Beverage and Retail, based in Beirut with a \$10,000–\$60,000 range, Gemini scored an in-niche, in-budget, in-region business at 96–98%, explicitly citing the matching niche, location, and budget fit. An out-of-niche, over-budget business in a different region scored 20–22%, with the explanation naming the specific mismatches rather than returning a generic low score.

This attribute-level grounding — rather than a plausible-sounding but non-specific justification — was the qualitative bar the project aimed for and, on manual inspection across multiple demo accounts, consistently met. Six sample investor profiles and six sample business profiles, spanning every niche and region in the schema with procedurally generated placeholder imagery, were seeded via a reproducible script to support demonstration without manually created test data.

5 Discussion & Analysis

The results support the project’s core hypothesis: a general-purpose, pretrained LLM can perform a useful compatibility-ranking task through prompting alone, without any task-specific training, provided its output is constrained to a structured schema. The schema constraint

proved essential rather than optional — it removes an entire class of failure (malformed or unparseable model output) that would otherwise require brittle regex extraction or retry logic, and it is what makes an LLM practical as a backend service dependency rather than a chat novelty.

The approach has clear limitations, most stemming from its scope as a small, unfunded project rather than from the method itself:

- 1. No formal evaluation.** No held-out set of investor–business pairs with human-assigned ground-truth compatibility exists, so claims about match quality rest on qualitative inspection rather than a measured precision or agreement score.
- 2. Retrieval bias.** The niche/region bias that keeps API costs bounded also means the AI never sees candidates outside that bias in the primary deck — a limitation the Browse directory only partially compensates for, since it requires the user to actively switch discovery modes.
- 3. Free-tier dependency.** Under sustained testing, the rule-based fallback engaged more often than production use would tolerate, and its simpler scoring is a genuine reduction in match quality, not a cosmetic one.
- 4. No real users.** The platform has only been exercised with synthetic seed data and the developer’s own test accounts — no real diaspora investor or Lebanese business has used it, so there is no evidence yet that its matches are useful or trustworthy to the people it is meant to serve.

These limitations point directly to future work: constructing even a small human-labeled evaluation set would let match quality be measured rather than asserted; a feedback loop that feeds actual swipe and match outcomes back into the scoring prompt (or a fine-tuned model, at greater scale) could close the gap between plausible-sounding and empirically good matches; and a genuine study with a small number of real diaspora investors would be the most important next step before treating this as more than a proof of concept.

6 Reflection on Learnings

The most valuable technical takeaway was learning to treat a hosted LLM as a constrained backend dependency rather than an open-ended chat feature: designing the prompt and schema together, batching requests for cost, and — critically — building a deterministic fallback so a rate-limited or unavailable API degrades the product instead of breaking it. This is a different discipline from prompting a chat interface, and it was the single most transferable skill from the project. Building the real-time layer correctly was a close second: getting authorization right at the room level, not just the REST layer, and separating per-match rooms from per-user

notification rooms taught me to think about real-time systems as a set of independently authorized channels rather than one open pipe.

The most rewarding moment was seeing the AI's explanations come back genuinely grounded in the specific profile data — citing the actual niche, region, or budget mismatch rather than generic filler — because that was the exact bar the project was trying to clear, and it was not guaranteed to happen just because the API call succeeded.

The most instructive challenge was a bug where seeded business images silently failed to render: business names and industry categories containing an ampersand (e.g. “Food & Beverage”) were embedded unescaped into procedurally generated SVG placeholder images, producing invalid XML that browsers refused to rasterize even though the server returned a valid response. Diagnosing it required moving past the assumption that a successful HTTP response meant a successful render, and instead inspecting the actual rendered image and the raw SVG source to find the real cause. It was a useful reminder that “the request succeeded” and “the feature works” are different claims, and that visual, browser-level verification — not just API testing — is necessary before calling a UI change complete.